

B1229062-MCE-Lab6-Result

問題&心得:

- (1) 在這次實驗遇到的問題比較少，第一個遇到的是要蜂鳴器無法撥出正確的音頻，他會發出的是持續的嘟嘟聲或是一段音高一樣的持續長因，後面經過一系列的排查，原因一是因為我們的程式撰寫蜂鳴器的腳位寫錯(P0.0 寫成 P1.0)，以及GND接線也插錯，然後在程式碼部分則是音符的頻率太快了，以上問題導致一開始的音樂播放有問題。
- (2) 第二個問題是我們在Bonus 1+2的部分，音樂在做停頓時會有吃掉音符、甚至是短休止以及長休止部分會互相干擾的問題，後面在程式管理停頓部分將原本分開做停頓的架構改成一區塊程式碼做統一管理才解決這個問題。
- (3) 最後一個問題是在做Bonus 2時，整首歌全部播完後蜂鳴器會有很小聲的持續嘟嘟聲，經過排查後，是要在程式最後面加上強制停止所有播音才解決(如果沒有停止的話，蜂鳴器依舊還在播音，但因為已經播完該播的音樂了，所以會播出持續的雜音)。

本次實驗內容結果:

- 1. lab06基本:(我們選的歌曲是快樂頌)
- (1) 作法:
 - 本次基本版主要是利用Timer0去產生不同頻率的方波，透過改變TH0/TL0的重載值來得到正確的音高，並讓蜂鳴器輸出對應旋律。
 - 首先把<<快樂頌>>的前四小節音符整理成Song[]，每一個音都以對應的 half-period(Timer0 的重載值)方式儲存，確保音高正確，再將每個音符的長短整理成 Dur[]，用簡單的 delay 方式控制音符時間長短(例如:短音、稍長、延音等)。
 - Play() 函式負責播放一個音:重複設定TH0/TL0，會開啟Timer0，再等待TF0溢位，最後flip蜂鳴器腳位，藉由快速翻轉產生方波。
 - 主程式以 for 迴圈依序讀取 Song[i]，每放完一個音符後會先停音，再依照 Dur[i] 做對應長度的延遲，使整段旋律能依照正確節奏播放。
- (2) 程式撰寫重點:
 - a. Timer0設定為 mode1(16-bit)，利用重載方式產生不同週期，對應DO、RE、MI等音符的頻率。
 - b. Play()採用重複溢位+翻轉蜂鳴器的方式輸出方波，讓音高的準確度由Timer0控制，而不是單純delay。
 - c. Dur[] 用於定義音符時值，Delay_ms(50 × Dur[i]) 則可以簡單做出不同音長，適可以先確認節奏正確。
 - d. Song[]與Dur[] 兩個陣列同步操作，確保每一個音符的音高與時間長短一一對應，不會錯位。
 - e. 主迴圈不使用中斷切換音符，而是以阻塞式播放整段旋律，整段期間CPU會無法做其他工作。
- (3) 結果:
 - a. 蜂鳴器能正確依照Song[] 播出前四小節旋律，音高正常、節奏清楚。
 - b. 不同音長(短音/中音/稍長)可明顯分辨，Dur[]的設定有正確作用。
 - c. 因為是delay阻塞方式，播放期間無法做其他工作，但旋律能順利完成。
 - d. 每段旋律結束後的停頓可辨識，讓歌曲聽起來不會黏在一起。
 - code:

```
#include <REG51.H> // 8051標頭檔

sbit BZ = P0^0; // 蜂鳴器接在P0.0

// 22.1184 MHz對應到的音符重載值
#define DO 0xF22A // Do
#define RE 0xF3C2 // Re
#define MI 0xF536 // Mi
#define FA 0xF5FD // Fa
#define SO 0xF729 // So
#define LA 0xF7D0 // La

// 四小節旋律(用Timer0重載值表示)
unsigned int code Song[] = {
    MI,MI,FA,SO,
    SO,FA,MI,RE,
    DO,DO,RE,MI,
    MI,RE,RE,

    MI,MI,FA,SO,
    SO,FA,MI,RE,
    DO,DO,RE,MI,
    RE,DO,DO
};

// 每個音要維持多久(倍率)
unsigned char code Dur[] = {
    2,2,2,3,
    3,2,2,2,
    2,2,2,3,
    2,2,3,

    2,2,2,3,
    3,2,2,2,
    2,2,2,3,
    2,3,4
};

// 簡單延遲(大概毫秒)
void Delay_ms(unsigned int t) {
    unsigned int i,j;
    for(i=0; i<t; i++)
        for(j=0; j<120; j++); // 空轉浪費時間
}

// 播放一個音(靠Timer0重複溢位 + 翻轉腳位)
void Play(unsigned int p) {
    unsigned int i;
    for(i=0; i<200; i++) { // 這個迴圈決定音大概播多久
        TH0 = p >> 8; // 重載值高位
        TL0 = p & 0xFF; // 重載值低位
        TR0 = 1; // 開Timer0
        while(!TF0); // 等溢位
        TF0 = 0; // 清溢位
        BZ = ~BZ; // 翻轉一次就是方波
    }
}
```

```
// 主程式
void main() {
    unsigned char i;
    TMOD = 0x01; // Timer0 mode1(16-bit)

    while(1) { // 歌曲重播
        for(i=0; i<sizeof(Song)/2; i++) {
            Play(Song[i]); // 播音符
            BZ = 0; // 播完先靜一下
            Delay_ms(50 * Dur[i]); // 用倍率控制音的長度
        }
        Delay_ms(400); // 一段播完的停頓
    }
}
```

2. bonus 1、2(以下內容是重點部分):

- (1) 作法:
 - a. bonus1主要把Basic版的delay全部改掉，改用Timer0 + Timer1雙中斷方式播音樂，Timer0專門產生音高，Timer1控制每個音符的時間與換音
 - b. 每一個音符會依Song[]放入對應的Timer0重載值，Timer0_ISR每次溢位時翻轉蜂鳴器腳位，使方波頻率正確。
 - c. 節奏全部交給Timer1_ISR計算，Timer1則是每約40ms進一次中斷，然後用tick累積時間，決定音符何時結束、何時換下一個。
 - d. bonus2又在bonus1的架構上加入休止(停頓)控制，分成兩個:
 - 短休止 **shortRest**:每個音與音之間的微停頓(讓分界更明顯)。
 - 長休止 **longRest**:每一句旋律結束時的較長停頓(句與句之間更自然)。
 - e. Timer1_ISR會依照tick判斷現在是在在播音、短休止、或長休止，並依序切換idx(音符位置)，全程不會中斷主程式。
- (2) 程式撰寫重點(不同於basic):
 - a. Timer0模式:固定用mode1，並在ISR中不停重載TH0/TL0，使音高能穩定。
 - b. Timer1 是音符控制的核心，依 ick左右變化做三件事:音符播完，進入shortRest ; shortRest播完，換下一音與重載Timer0 ; 每句末尾=>longRest。
 - c. 新增shortRest與longRest兩個旗標，避免音樂休止時還在翻轉蜂鳴器，讓休止後真正靜音，不在播出任何聲音。

- d. 每次換音或進入休止時，都會先把BZ清成0，以免殘留雜音。
- (3) 結果:本次Bonus1+2以雙中斷方式播放完整旋律，並加入短/長休止讓音樂播放比basic版更清楚、段落更自然。
- code:

```
#include <REG51.H> // 8051的暫存器定義

sbit BZ = P0^0; // 蜂鳴器在P0.0

// 不同音符的Timer0重載值(22.1184MHz算出來的)
#define DO 0xF22A
#define RE 0xF3C2
#define MI 0xF536
#define FA 0xF5FD
#define SO 0xF729
#define LA 0xF7D0

// 樂譜:依序存音符
unsigned int code Song[] = {
    MI,MI,FA,SO,
    SO,FA,MI,RE,
    DO,DO,RE,MI,
    MI,RE,RE,

    MI,MI,FA,SO,
    SO,FA,MI,RE,
    DO,DO,RE,MI,
    RE,DO,DO
};

// 每個音要維持多久(越大越長)
unsigned char code Dur[] = {
    2,2,2,2,
    2,2,2,2,
    2,2,2,2,
    2,2,4,

    2,2,2,2,
    2,2,2,2,
    2,2,2,2,
    2,3,4
};

unsigned char idx = 0; // 播到第幾個音
unsigned char tick = 0; // Timer1在用的時間累積
bit shortRest = 0; // 音與音之間的小停頓
bit longRest = 0; // 每句之間的較長停頓

// Timer0:發出方波(控制音高)
void Timer0_ISR(void) interrupt 1 {
    if(!shortRest && !longRest) // 休止時不發聲
        BZ = ~BZ;

    TH0 = Song[idx] >> 8; // 載入目前音符(高位)
    TL0 = Song[idx] & 0xFF; // 載入目前音符(低位)
}

// Timer1:負責節奏+換音+休止
void Timer1_ISR(void) interrupt 3 {
    tick++; // 每40ms+1(大約節拍器)

    // 播完一個音，進入短休止
    if(!shortRest && !longRest && tick >= Dur[idx]*2) {
        shortRest = 1; // 開短休止
        tick = 0;
        BZ = 0; // 先靜音
        return;
    }
}

// 短休止結束，換下一個音
if(shortRest && tick >= 1) { // 1*40ms
    shortRest = 0;
    tick = 0;
    idx++; // 換音

    // 每句結束後，進入長休止
    if(idx==4 || idx==8 || idx==12 || idx==15 ||
       idx==19 || idx==23 || idx==27 || idx==30) {
        longRest = 1; // 開長休止
        BZ = 0;
        return;
    }
}

// 播到最後了
if(idx >= sizeof(Song)/2) {
    TR0 = TR1 = 0; // 停Timer
    ET0 = ET1 = 0; // 關中斷
    BZ = 0; // 完全靜音
    return;
}

// 載入下一個音的重載值
TH0 = Song[idx] >> 8;
TL0 = Song[idx] & 0xFF;
return;
}

// 長休止(句與句之間)
if(longRest && tick >= 6) { // 約240ms
    longRest = 0;
    tick = 0;

    if(idx >= sizeof(Song)/2) {
        TR0 = TR1 = 0;
        ET0 = ET1 = 0;
        BZ = 0;
        return;
    }
}

// 長休止完繼續下一句
TH0 = Song[idx] >> 8;
TL0 = Song[idx] & 0xFF;
return;
}

// 主程式:開Timer就好，其餘交給中斷
void main() {
    TMOD = 0x11; // Timer0/Timer1都用mode1

    TH0 = Song[0] >> 8; // 先載入第一個音
    TL0 = Song[0] & 0xFF;

    TH1 = 0x3C; // Timer1每 ~40ms進一次中斷
    TL1 = 0xB0;

    ET0 = ET1 = EA = 1; // 開中斷
    TR0 = TR1 = 1; // 開Timer

    while(1); // 主程式不做事，音樂靠中斷播
}
```

- 實驗照片:

